

AWS SYSTEMS MANAGER – PARAMETER STORE

Benefits of Using AWS Services

AWS SQS (Simple Queue Service)

Key Benefits

- Decouples applications so producer and consumer work independently
- Enables asynchronous processing, improving performance
- Highly reliable (messages are not lost)
- Automatically scales with traffic
- Supports DLQ to handle failed messages

Business Value

- Prevents application crashes during traffic spikes
 - Improves fault tolerance and resilience
-

Amazon MQ

Key Benefits

- Supports traditional messaging protocols (JMS, AMQP, MQTT)
- Ideal for legacy application migration to AWS
- Provides managed ActiveMQ/RabbitMQ without operational overhead
- Supports high availability (Multi-AZ)
- Ensures guaranteed message delivery

Business Value

- Enables hybrid and enterprise messaging systems
 - Reduces infrastructure management effort
-

AWS Systems Manager Parameter Store

Key Benefits

- Centralized storage for configuration and secrets
- Supports SecureString encrypted with KMS
- Eliminates hardcoded credentials
- Fine-grained access using IAM
- Fully auditable with CloudTrail

Business Value

- Improves security and compliance
 - Simplifies configuration management across environments
-

AWS WAF (Web Application Firewall)

Key Benefits

- Protects applications from OWASP Top 10 attacks
- Provides rate limiting to prevent DDoS attacks
- Integrates with ALB, API Gateway, CloudFront
- Custom and managed rule support
- Real-time monitoring and logging

Business Value

- Enhances application security
- Prevents data breaches and service abuse

1. SSM PARAMETER STORE – TASK 1

Store Application Configuration (String Parameter)

STEPS I FOLLOWED (You can say this confidently)

1. Opened AWS Systems Manager → Parameter Store and created a new parameter.

2. Used hierarchical naming /prod/app/db_host to separate environment and application.
3. Chose String type since the value is non-sensitive.
4. Created a custom IAM policy allowing ssm:GetParameter on /prod/app/*.
5. Attached the policy to an EC2 IAM role and associated the role with the EC2 instance.
6. Verified access by retrieving the parameter using AWS CLI from EC2.

Created parameter store AWS system manger > parameter store < create parameter

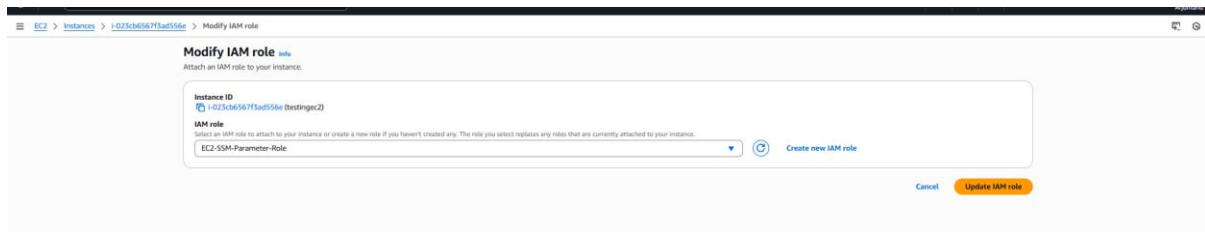
The screenshot shows the AWS Systems Manager Parameter Store console. The main heading is "AWS Systems Manager Parameter Store" with the subtitle "Secrets and configuration data management". Below this, there's a "Start to use Parameter Store" button and a "Create parameter" button. The "How it works" section lists two steps: "Create a new parameter" and "Specify the parameter type and values". The "Create parameter" form is open, showing the following details:

- Name:** /prod/app/db_host
- Description — Optional:**
- Tier:** Standard (selected). Advanced is also an option.
- Type:** String (selected). Other options are StringList and SecureString.
- Data type:** text
- Value:** prod-db.internal
- Tags — Optional:**

Created a custom IAM policy allowing ssm:GetParameter on /prod/app/*.

The screenshot shows the AWS IAM console "Create role" wizard, specifically the "Add permissions" step. The "Permissions policies" section shows a search for "SSM-Read-Prod-App" with 1 match. The selected policy is "SSM-Read-Prod-App" with the description "Allows EC2 to read SSM Parameter Store v...". The "Set permissions boundary - optional" section is collapsed. The "Next" button is highlighted.

Created role and attached that policy which I have created



Role is added to ec2 and connected ec2 and checked using the command 'aws ssm get-parameter --name /prod/app/db_host' after aws configure

```
/opt/  
Last login: Thu Dec 18 11:59:29 2025 from 10.206.107.29  
[ec2-user@ip-12-0-0-167 ~]$ aws configure  
AWS Access Key ID [*****]: AWS Secret Access Key [*****]: Default region name [us-east-1]: Default output format [json]:  
[ec2-user@ip-12-0-0-167 ~]$ aws ssm get-parameter --name /prod/app/db_host  
{  
  "Parameter": {  
    "Name": "/prod/app/db_host",  
    "Type": "String",  
    "Value": "prod-db.internal\n",  
    "Version": 1,  
    "LastModifiedDate": "2025-12-19T09:17:54.497000+00:00",  
    "ARN": "arn:aws:ssm:us-east-1:679625722057:parameter/prod/app/db_host",  
    "DataType": "text"  
  }  
}
```

2. SSM Parameter Task 2: Secure Secrets Using SecureString + KMS

Business Scenario

Passwords and tokens must be encrypted.

Task

1. Create *SecureString* parameter
2. Encrypt using *Customer-Managed KMS key*
3. Restrict access via IAM policy
4. Retrieve decrypted value in EC2/Lambda
5. Audit access using CloudTrail

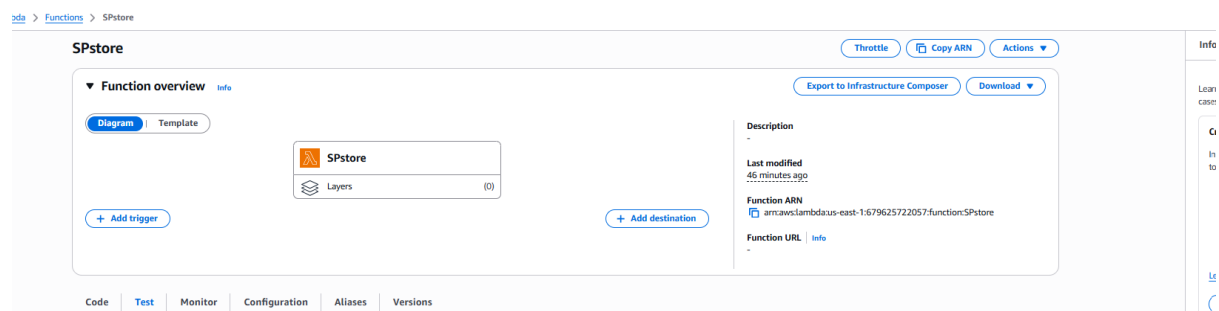
Steps:

SSM Parameter Task 2: Secure Secrets Using SecureString + KMS (10 Steps)

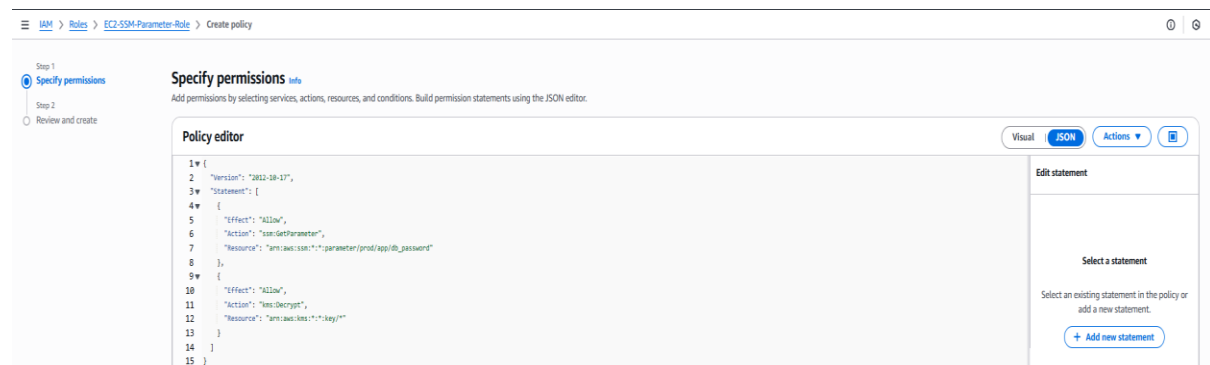
1. Created a **Customer-Managed KMS key** and enabled automatic rotation.
2. Opened **AWS Systems Manager → Parameter Store**.
3. Created a parameter named **/prod/app/db_password**.
4. Selected **SecureString** as the parameter type.

5. Chose the **customer-managed KMS key** for encryption.
6. Saved the secret value securely in Parameter Store.
7. Created a **Lambda execution IAM role** with least-privilege permissions.
8. Attached permissions for **ssm:GetParameter** and **kms:Decrypt**.
9. Updated the **KMS key policy** to allow the Lambda role to decrypt.
10. Tested the Lambda and verified success in **CloudWatch Logs**.

Created a lambda function



“The IAM role defines exactly what my Lambda function can do. It allows Lambda to read a specific SecureString parameter from SSM, decrypt it using KMS, and write logs to CloudWatch—nothing more.”



The KMS key policy controls *who is allowed to use the encryption key itself*.

Even if IAM allows an action, **KMS will deny it unless the key policy also allows it**.

So, this policy is the **final gatekeeper**.

97224261-3e1d-4d5a-8c8a-637b88a5f037

Key actionsEdit

General configuration

Alias
alias/s3-pii-key

ARN
arn:aws:kms:us-east-1:679625722057:key/97224261-3e1d-4d5a-8c8a-637b88a5f037

Current key material ID
d1083a984828027d4ce78872142997c5d5723f04d61cab3d8bc7ba2aa3dfdd
b7

Status
Enabled

Description
-

Creation date
Dec 18, 2025 13:09 GMT+5:30

Regionality
Single Region

Key policyCryptographic configurationTagsKey material and rotationsAliases

Key policy

Edit

```
11  "Resource": "*",
12  },
13  {
14    "Sid": "AllowLambdaDecrypt",
15    "Effect": "Allow",
16    "Principal": {
17      "AWS": "arn:aws:iam::679625722057:role/service-role/SPstore-role-bsavp9k2"
18    },
19    "Action": [
20      "kms:Decrypt",
21      "kms:DescribeKey"
22    ],
23    "Resource": "*"
24  }
25 ]
26 }
```

This proves **everything** worked perfectly:

- ✓ Lambda executed
- ✓ IAM role permissions are correct
- ✓ KMS key decrypted successfully
- ✓ SSM SecureString was retrieved
- ✓ Logs written to CloudWatch

👉 **Your SSM Parameter Task 2 is 100% COMPLETE**

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear1m30m1h12hCustomUTC timezoneDisplay

Timestamp

Message

No older events at this moment. [Retry](#)

▶ 2025-12-21T18:39:08.675Z
INIT_START Runtime Version: python:3.10.v209 Runtime Version ARN: arn:aws:lambda:us-east-1:runtime:c3d97813172c48429cac556b761d440001b617eb86dcf2dca3854ba843bc573

▶ 2025-12-21T18:39:08.958Z
START RequestId: ec751371-a434-4f01-aebf-44f6091d5354 Version: \$LATEST

▶ 2025-12-21T18:39:08.365Z
SecureString retrieved successfully

▶ 2025-12-21T18:39:08.398Z
END RequestId: ec751371-a434-4f01-aebf-44f6091d5354

▶ 2025-12-21T18:39:08.398Z
REPORT RequestId: ec751371-a434-4f01-aebf-44f6091d5354 Duration: 2638.82 ms Billed Duration: 2729 ms Memory Size: 128 MB Init Duration: 279.31 ms

No newer events at this moment. [Auto retry paused](#) [Resume](#)

“I stored application secrets in SSM Parameter Store using SecureString encrypted with a customer-managed KMS key and validated secure access from Lambda using IAM roles.”

3. Setup SQS and store messages to sqs by lambda auto trigger and use s3 bucket store messages

Created an S3 Bucket

- Created an S3 bucket (mysqsbucket01arj) to store objects
- Bucket is private and used as the event source
- Purpose: trigger notifications whenever a new object is uploaded

The screenshot shows the AWS S3 console for the bucket 'mysqsbucket01arj'. The 'Properties' tab is selected, displaying the following information:

- Bucket overview:**
 - AWS Region: US East (N. Virginia) us-east-1
 - Amazon Resource Name (ARN): arn:aws:s3:::mysqsbucket01arj
 - Creation date: December 19, 2025, 20:47:23 (UTC+05:30)
- Bucket Versioning:** Enabled. A link to 'Learn more' is provided.
- Multi-factor authentication (MFA) delete:** Disabled. A link to 'Learn more' is provided.
- Bucket ABAC:** Disabled. A link to 'Learn more' is provided.
- Tags:** A section explaining that tags can be used to track storage costs and specify permissions. It includes a link to 'Learn more'.

Created an SQS Standard Queue

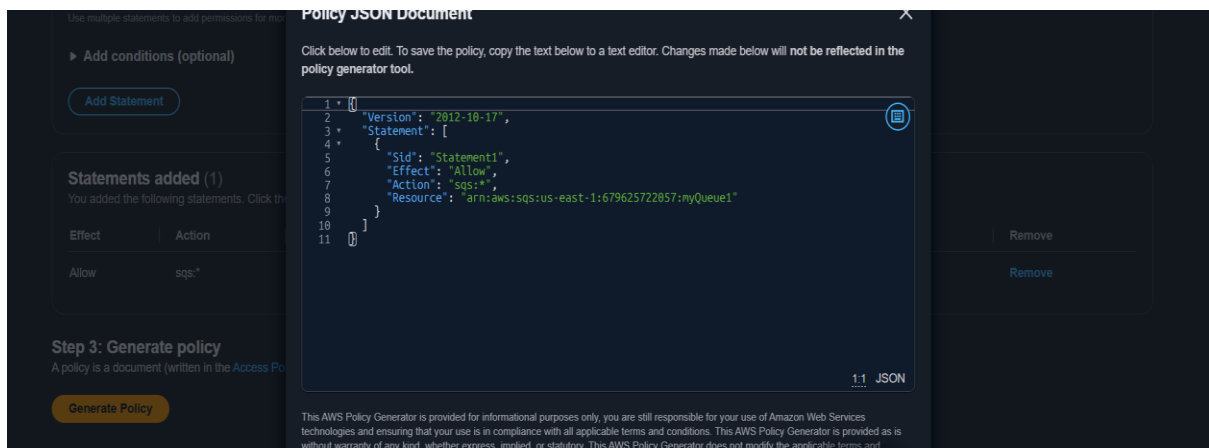
- Created an SQS standard queue (myQueue1)
- This queue is used to decouple S3 events from Lambda processing
- Ensures reliable, asynchronous message delivery

The screenshot shows the AWS IAM console 'Step 2: Add statement(s)' for creating a policy. The configuration is as follows:

- Type of Policy:** IAM Policy
- Effect:** Allow (selected)
- AWS:** All Services (***)
- Add conditions (optional):** A condition is added with the following details:
 - Condition:** ArnEquals
 - Key:** aws:CurrentTime
 - Value:** arn:aws:s3:::aws-lambda-mybucket1/134005378425871691.jpg

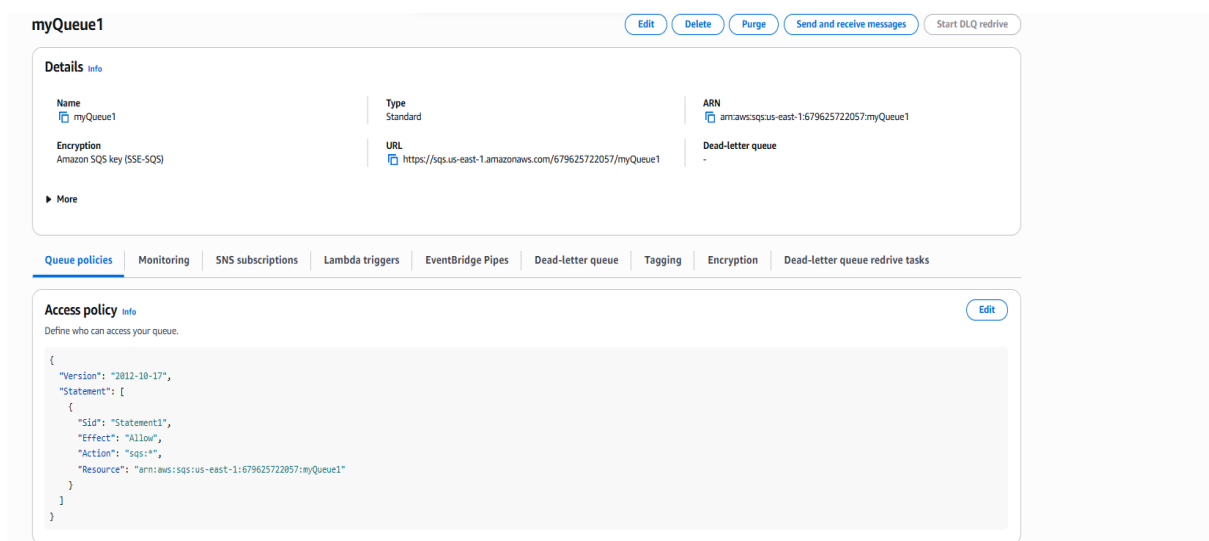
Updated SQS Access Policy (Critical Step)

- Edited the SQS queue policy to allow **only S3** to send messages
- Added a resource-based policy with:
 - Principal: s3.amazonaws.com
 - Action: sqs:SendMessage
 - Condition restricting access to the specific S3 bucket ARN
- This securely enables S3 → SQS communicatio



Configured S3 Event Notifications

- Enabled **ObjectCreated** events on the S3 bucket
- Configured the event destination as the SQS queue (myQueue1)
- Ensured that every new file upload sends a message to SQS



Created a Lambda Function

- Created a Lambda function using **Python runtime**
- Function purpose: process messages coming from SQS
- Lambda is used for serverless, scalable event processing

The screenshot shows the 'Create function' page in the AWS Lambda console. The 'Author from scratch' option is selected. The 'Function name' is 'sqs-s3-trigger'. The 'Runtime' is 'Python 3.14'. The 'Architecture' is 'x86_64'. The 'Permissions' section shows the default execution role. The 'Additional configurations' section is expanded, showing settings for networking, security, and governance. The 'Create function' button is visible at the bottom right.

Assigned IAM Permissions to Lambda

- Attached IAM permissions to allow Lambda to:
 - Receive messages from SQS
 - Delete messages after processing
 - Read queue attributes
- Ensured least-privilege access

The screenshot shows the 'Attach policy to sqs-s3-trigger-role-ug784jqj' page in the AWS IAM console. The 'Current permissions policies (1)' section shows the 'sqs-s3-trigger-role-ug784jqj' policy. The 'Other permissions policies (1/1117)' section shows a list of policies, including 'AmazonS3FullAccess' and 'AmazonS3ObjectLambdaExecutionRolePolicy'. The 'Filter by Type' dropdown is set to 'All types', and the '15 matches' are displayed.

Attached SQS as a Lambda Trigger

- Added the SQS queue as an event source (trigger) for Lambda
- Enabled the trigger so Lambda automatically runs when messages arrive
- Batch processing handled by Lambda service

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
myQueue1	Standard	2025-12-19T20:45:05:30	1	0	Amazon SQS key (SSE-SQS)	-

Implemented Lambda Python Code

- Lambda code parses the SQS message
- Extracts S3 bucket name and object key from the event payload
- Logs the file upload details to CloudWatch Logs

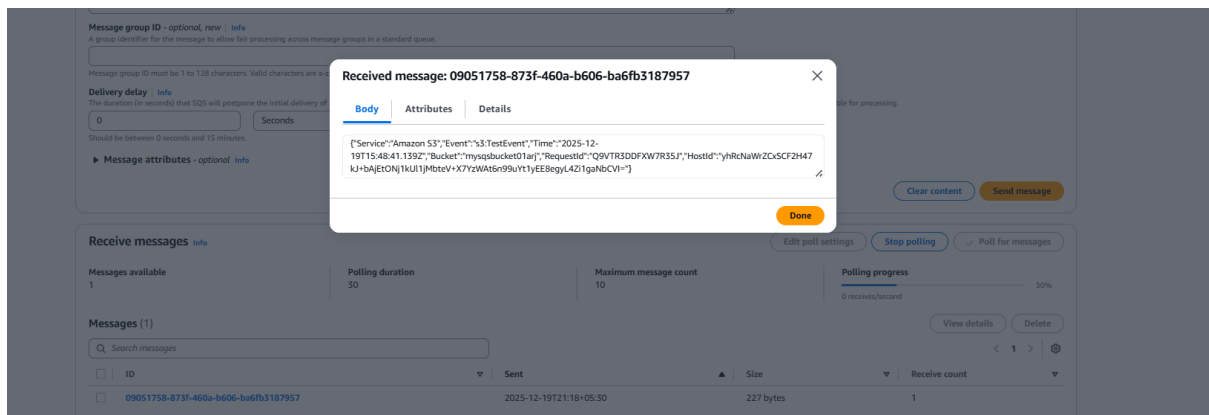
The screenshot shows the 'Destination' configuration page in the AWS IAM console. It includes a warning box about granting permissions to the Amazon S3 principal. Below, the 'Destination' section has three radio buttons: 'Lambda function', 'SNS topic', and 'SQS queue', with 'SQS queue' selected. The 'Specify SQS queue' section has two radio buttons: 'Choose from your SQS queues' (selected) and 'Enter SQS queue ARN'. A dropdown menu for 'SQS queue' shows 'myQueue1'. At the bottom right are 'Cancel' and 'Save changes' buttons.

Tested the End-to-End Flow

- Uploaded a file to the S3 bucket
- Verified message arrival in SQS
- Confirmed Lambda execution in CloudWatch Logs
- Ensured successful processing without errors

The screenshot shows the 'myQueue1' page in the AWS IAM console. It includes tabs for 'Details', 'Queue policies', 'Monitoring', 'SNS subscriptions', 'Lambda triggers', 'EventBridge Pipes', 'Dead-letter queue', 'Tagging', 'Encryption', and 'Dead-letter queue redrive tasks'. The 'Details' tab is active, showing the queue's name, type, encryption, and URL. A yellow arrow points to the 'Send and receive messages' button. Below, the 'SNS subscriptions' section shows a list of subscriptions with a 'Subscribe to Amazon SNS topic' button. At the bottom, the 'Receive messages' section shows a 'Poll for messages' button, which is highlighted with a yellow arrow.

Messages triggers and successfully message is displayed in SQS

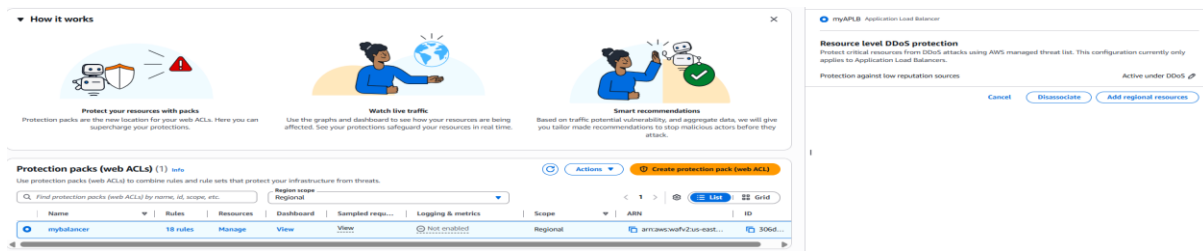


4. Create a WAF and attach resource as load balancer

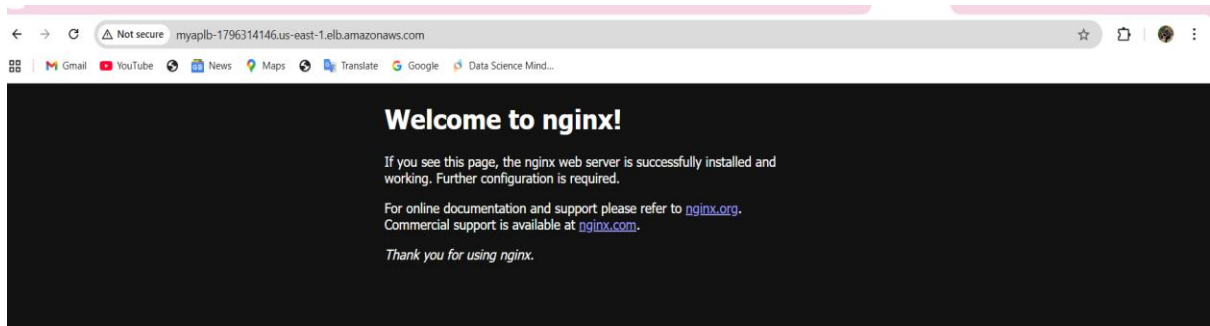
Steps Followed to Set Up AWS WAF with Application Load Balancer

1. Created an **Application Load Balancer (ALB)** and attached EC2 instances in a target group.
2. Verified ALB accessibility using the **DNS name**.
3. Opened **AWS WAF & Shield** and created a **Web ACL (Protection Pack)** with **Regional scope**.
4. Attached the Web ACL to the **Application Load Balancer**.
5. Created an **IP set** with the correct CIDR range (due to dynamic IP, used subnet range instead of /32).
6. Added a **Custom rule**:
 - Action: **Block**
 - Condition: **Originates from an IP address**
 - IP address list: Created IP set
7. Ensured the rule used **Source IP address** (not X-Forwarded-For header).
8. Placed the **Block rule at the top priority** so it evaluates before allow rules.
9. Adjusted rule order to ensure no **Allow rules override the Block rule**.
10. Tested access to ALB and confirmed blocking behavior based on WAF rules.

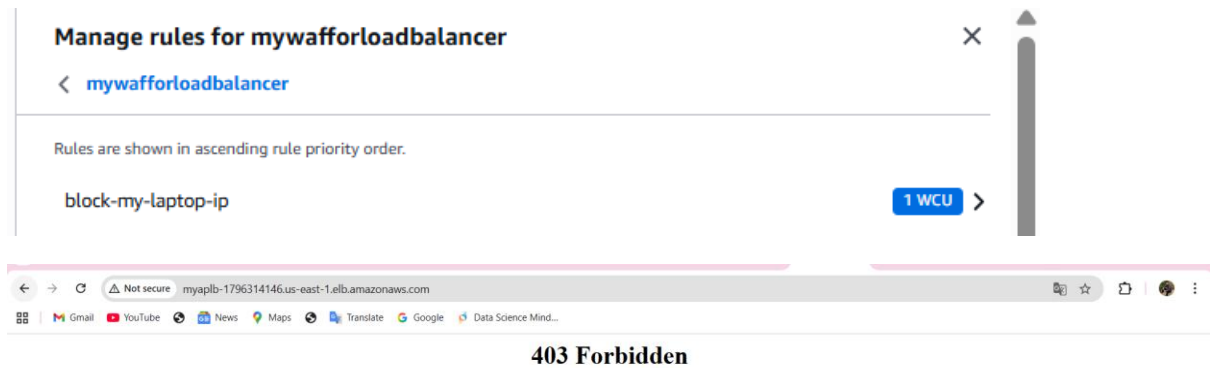
✅ Result: **AWS WAF successfully configured and protecting the Application Load Balancer.**



With help of WAF also I am able to access



Blocked now my IP address.



Now I have allowed my ip address again

block-my-laptop-ip

×

< Manage rules for mywafforloadbalancer

Visual | JSON

Action

Allow

Rule name

block-my-laptop-ip

Names must use 1-128 characters from A-Z, a-z, 0-9, hyphen, and underscore.

If a request

matches the statement

Inspect

Originates from an IP address in

Statement

IP address list

Block-my-laptop_IP

×

▶ Rule configuration

▶ Custom request - optional

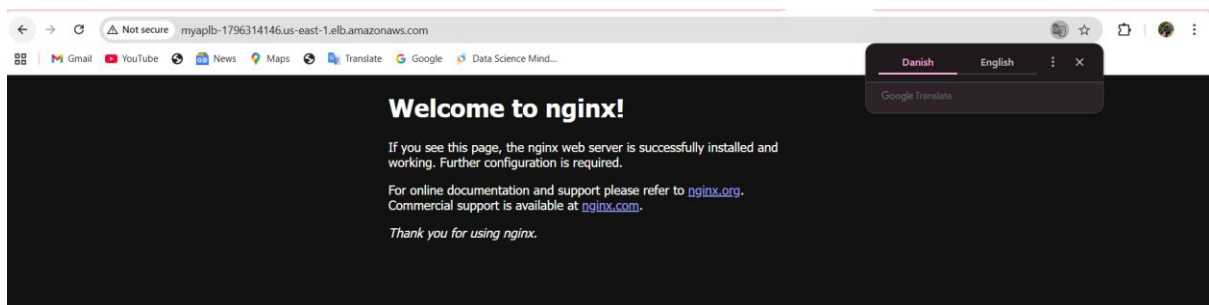
▶ Add labels - optional

Cancel

Delete

Save rule

Now again it is allowing so successfully attached firewall.



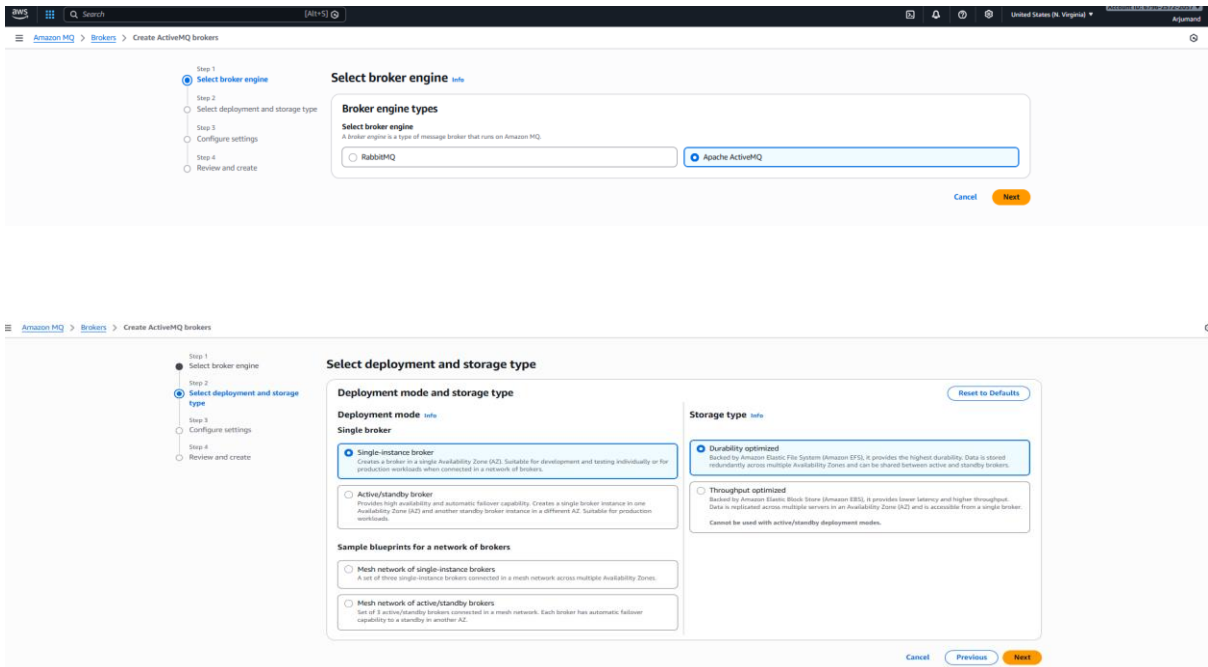
5. Create and setup MQ broker and login

Steps Followed to Set Up Amazon MQ (ActiveMQ)

1. Opened Amazon MQ in AWS Console and clicked Create broker.
2. Selected Apache ActiveMQ as the broker engine and chose Single-instance broker for learning.
3. Provided broker name, engine version, instance type, and created broker user credentials (username & password).
4. Selected a custom VPC (default VPC was deleted) and chose two subnets from different Availability Zones.

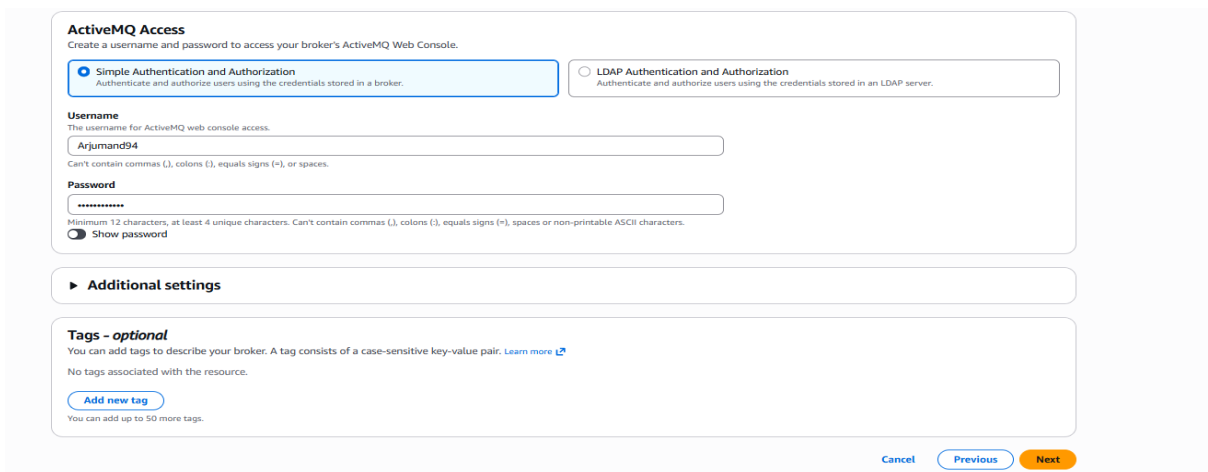
5. Attached a security group and allowed required ports (8162 for console, 61617 for OpenWire).
6. Created the broker and waited until the status changed to Running.
7. Opened the ActiveMQ Web Console URL using port 8162.
8. Logged in using the broker credentials (not IAM).
9. Verified successful setup by accessing the ActiveMQ dashboard and broker details.

 **Result: Amazon MQ broker setup and access completed successfully.**



The screenshot displays the AWS Management Console interface for creating an Amazon MQ broker. The breadcrumb trail shows 'Amazon MQ > Brokers > Create ActiveMQ brokers'. The left sidebar contains a progress indicator with four steps: 'Step 1: Select broker engine' (active), 'Step 2: Select deployment and storage type', 'Step 3: Configure settings', and 'Step 4: Review and create'. The main content area is titled 'Select broker engine' and includes a 'Broker engine types' section with a search bar and two radio buttons: 'RabbitMQ' and 'Apache ActiveMQ' (selected). Below this is the 'Select deployment and storage type' section, which is divided into 'Deployment mode and storage type' and 'Storage type'. Under 'Deployment mode and storage type', there are three radio buttons: 'Single broker' (selected), 'Active/standby broker', and 'Sample blueprints for a network of brokers'. Under 'Storage type', there are two radio buttons: 'Durability optimized' (selected) and 'Throughput optimized'. The 'Next' button is highlighted in orange.

Provided username and password



The screenshot shows the 'ActiveMQ Access' configuration page. The 'Simple Authentication and Authorization' option is selected, with a description: 'Authenticate and authorize users using the credentials stored in a broker.' The 'LDAP Authentication and Authorization' option is also visible. The 'Username' field contains 'Arjumand94'. The 'Password' field is masked with dots. Below the password field, there is a 'Show password' toggle. The 'Additional settings' section is expanded, showing 'Tags - optional'. The 'Tags' section states: 'You can add tags to describe your broker. A tag consists of a case-sensitive key-value pair. Learn more'. There is an 'Add new tag' button. The 'Next' button is highlighted in orange.

Selected default VPC and subnet

Network settings

Access type | Info

☒ **Public access**
The broker can be accessed directly, outside of a VPC.

☐ **Private access**
A broker instance that isn't publicly accessible can be accessed only within a VPC.

VPC and subnets

☐ **Use the default VPC and subnet(s)**
Amazon MQ automatically selects the VPC and subnet(s) for you.

☒ **Select existing VPC and subnet(s)**

VPC | Info
The VPC defines the virtual networking environment for this broker.

vpc-0018a135ed108a41b My_vpc_virginaR [Create a new VPC](#)
Connection method: IPv4

Subnet(s) | Info
A segment of the IP address range of the selected VPC to which you can attach EC2 instances. One subnet is required for a single-broker deployment.

subnet-023fb7044fd931fb8 pri_subnet.N1 [Create a new subnet](#)
Availability zone: us-east-1a Connection method: IPv4

Security group(s) | Info
Security groups define the rules that authorize connections from all EC2 instances and devices that require access to your broker instance.

☒ **Use the default security group**
Amazon MQ automatically selects the default security group for selected VPC.

☐ **Select existing security groups**
Select an existing security group with inbound and outbound rules for your broker.

Data encryption

Encryption
You can use AWS Key Management Service (KMS) to create and manage customer master keys (CMKs). Amazon MQ uses CMKs to encrypt your data at rest. [Learn more](#)

☒ **AWS owned CMK**
The key is owned by Amazon MQ and is not in your account.

☐ **AWS managed CMK**
The AWS managed CMK (aws/mq) is a CMK in your account that is created, managed, and used on your behalf by Amazon MQ.

☐ **Customer managed CMKs** are created and managed by you in AWS Key Management Service (KMS).

Maintenance

Maintenance window
During the maintenance window, Amazon MQ performs broker maintenance operations, such as automatic version upgrades

☒ **No preference**
Amazon MQ selects the maintenance window for you.

MQ created with default vpc and default subnet

Broker my-mq-broker is being created.
Broker creation takes about 20 minutes. Your broker's configuration can be found on the Configurations page.

Brokers for us-east-1 (1) [Edit](#) [Delete](#) [Create replica broker](#) [Create brokers](#)

Name	Status	Creation time (Local)	Broker engine	Deployment mode	Instance type	Data replication role
my-mq-broker	Creation in progress	December 21, 2025 at 13:56 (UTC+5:30)	Apache ActiveMQ	Single-instance broker	mq.m5.large	-

Edit security group of broker and allow tcp 8162 port

sg-07bc36e2a7e980569 - default > Edit inbound rules

Edit inbound rules | Info
Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules | Info
Security group rule ID: sgr-0c8459e0ab328bc5

Type	Protocol	Port range	Source	Description - optional
All traffic	All	All	Custom	
-	Custom TCP	TCP	8162	Anywh...

[Add rule](#) [Cancel](#) [Preview changes](#) [Save rules](#)

Click the link which is there in broker

Amazon MQ > [Brokers](#) > my-mq-broker

Amazon MQ

[Brokers](#)
[Configurations](#)

ActiveMQ
Broker engine version: 5.18.7
Creation time (Local): December 21, 2025 at 13:56 (UTC+5:30)

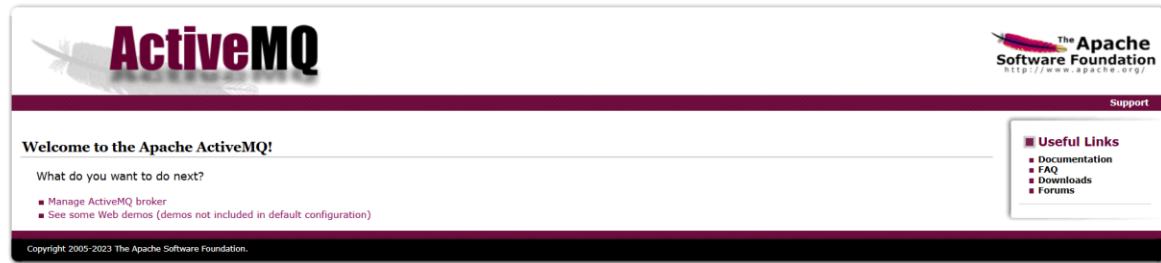
Connections
Access your queues and topics and connect your application to the broker. If you disable public accessibility for your broker, your endpoints are reachable only within a VPC.

Enable connections to your broker
To be able to access your broker's ActiveMQ Web Console URL, or wire-level protocol endpoints, you must configure one of your security groups to allow inbound traffic.

ActiveMQ Web Console
In an active/standby deployment, only one of the ActiveMQ Web Console URLs is active at a time.
[To be able to access your broker's ActiveMQ Web Console URLs is active at a time.](#)

Endpoints
/console/

After clicking the link we can see the broker page and login with the credentials which we created while creating MQ broker



After clicking Manage ActiveMQ it will ask credentials then enter then the page will open

